

Python Logic: Finding the Last Uppercase Word

2024 Sep OPPE 1 – Set 3 | Section 2 – Problem 1

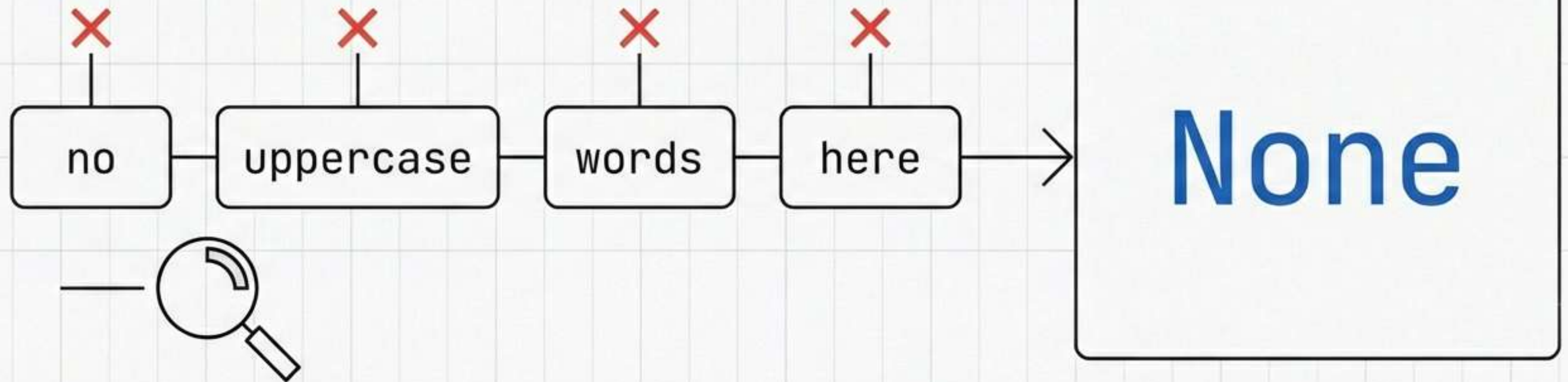
OBJECTIVE: COMPLETE THE FUNCTION DEFINITION. NO INPUT/OUTPUT HANDLING REQUIRED.

The Challenge



- ✓ Words separated by spaces
- ✓ **No punctuation handling** needed
- ✓ **Case-sensitive check** (first character only)

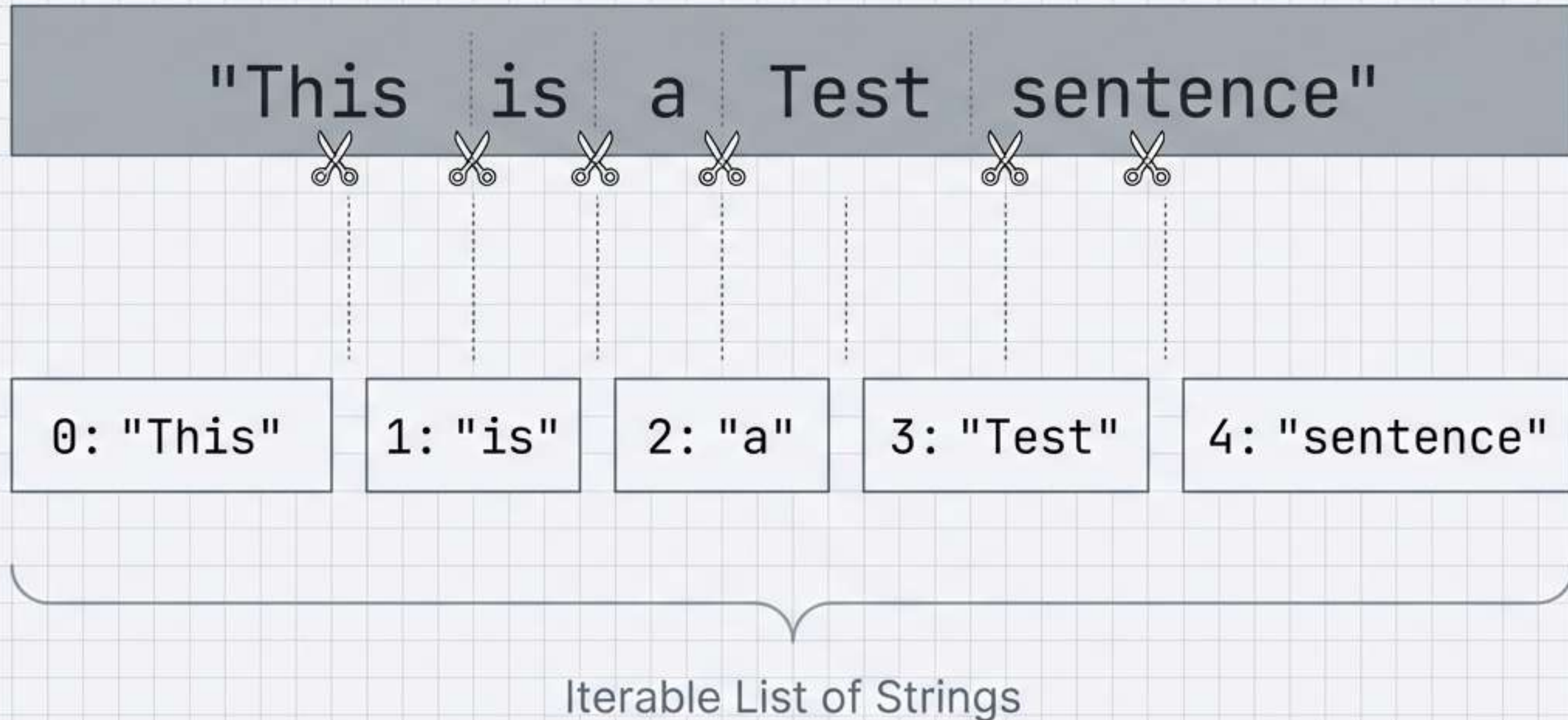
Handling the Edge Case



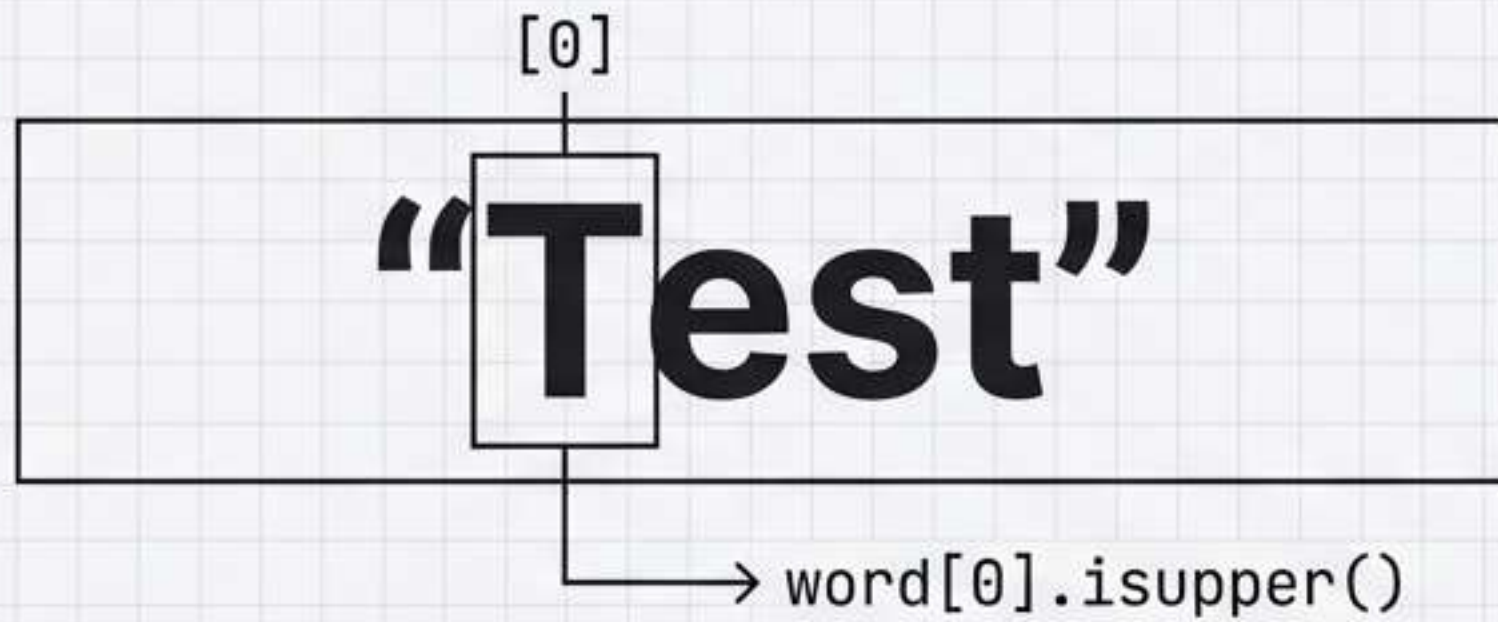
*If the loop finishes without a match, the function must return **None**.*

Step 1: Breaking the Chain

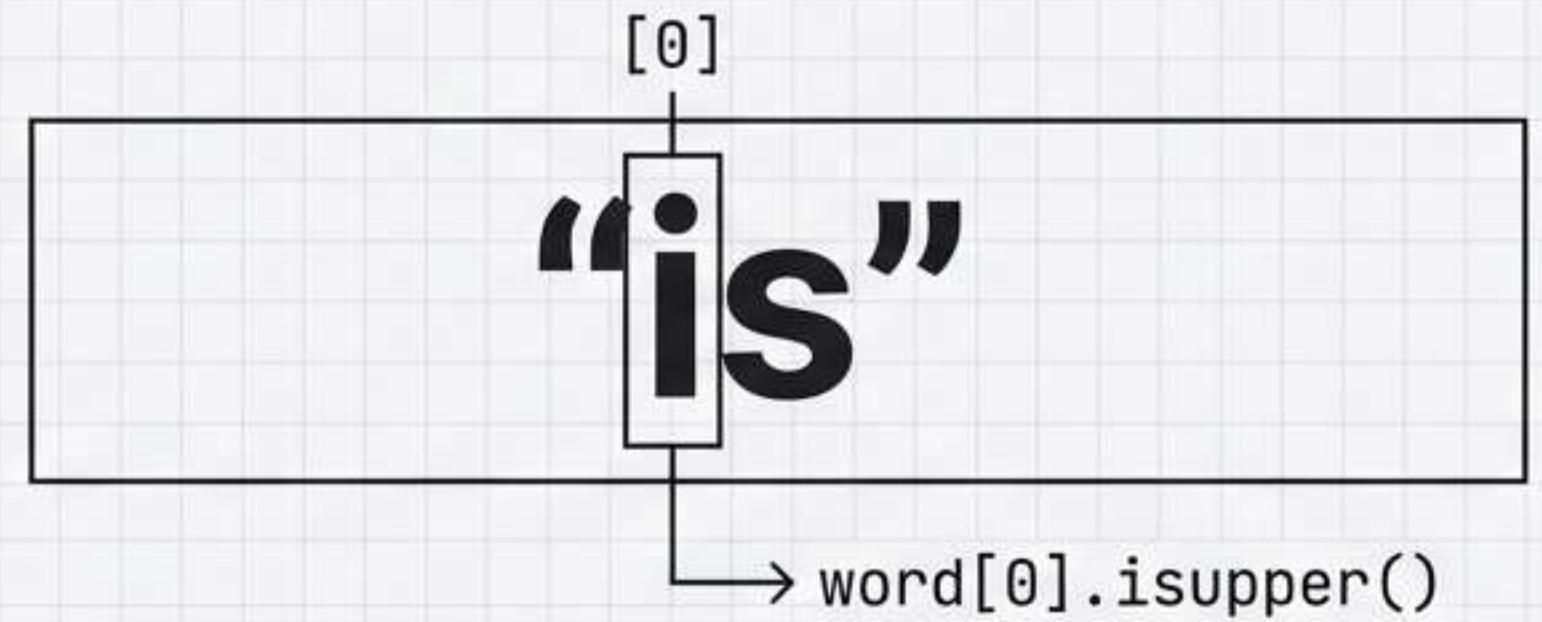
```
words = sentence.split()
```



Step 2: The Gatekeeper



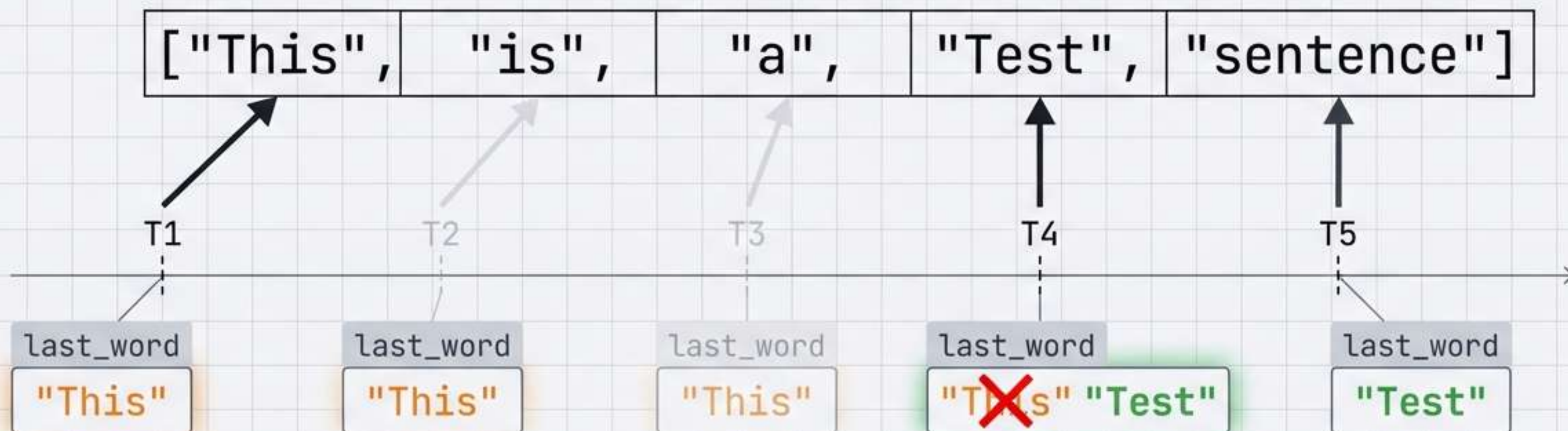
True



False

Logic is strictly focused on the 0th index.

Approach 1: The Forward March



Inefficiency: Requires scanning the entire list and overwriting variables.

This method iterates through every element, leading to unnecessary comparisons and redundant variable updates when a qualifying word is found late in the sequence.

Implementation: Forward Traversal

```
def last_word_starts_with_upper_case(sentence: str):  
    words = sentence.split()  
    last_word = None          # Initialize storage  
  
    for word in words:        # Scan Left -> Right  
        if word[0].isupper():  
            last_word = word  # Update repeatedly  
  
    return last_word
```

Overwrites
previous match

Is there a more efficient way?

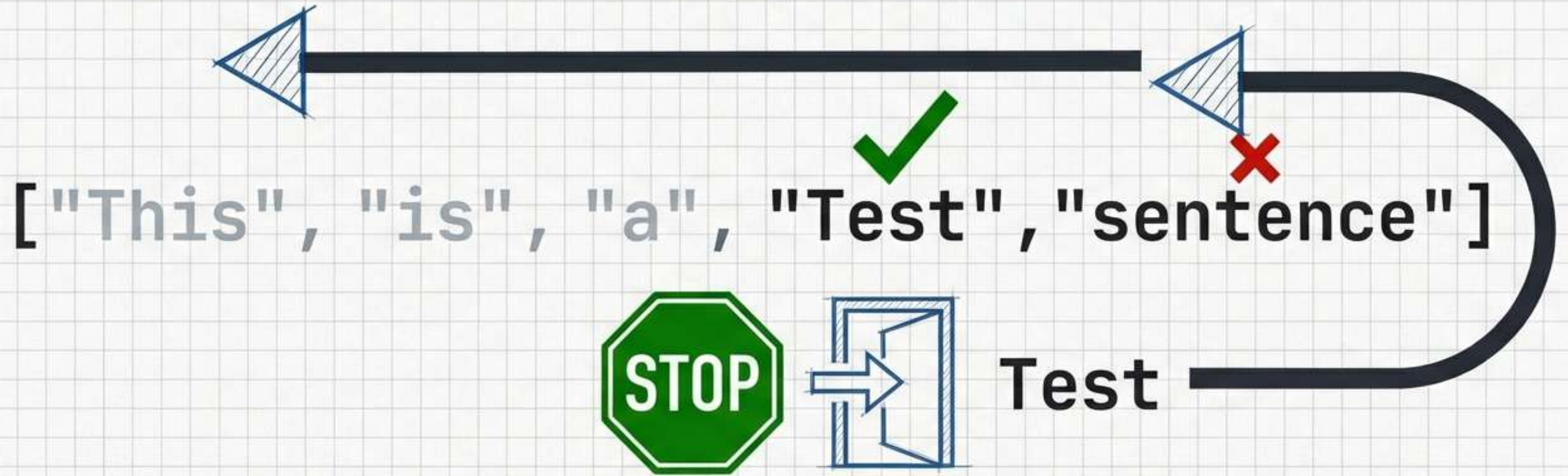
1 → 2 → 3 → 4 → 5

5 ← 4 — **STOP**

We are looking for the LAST occurrence.
Why start at the beginning?

reversed(words)

Approach 2: The Reverse Engineer



Early Exit Strategy

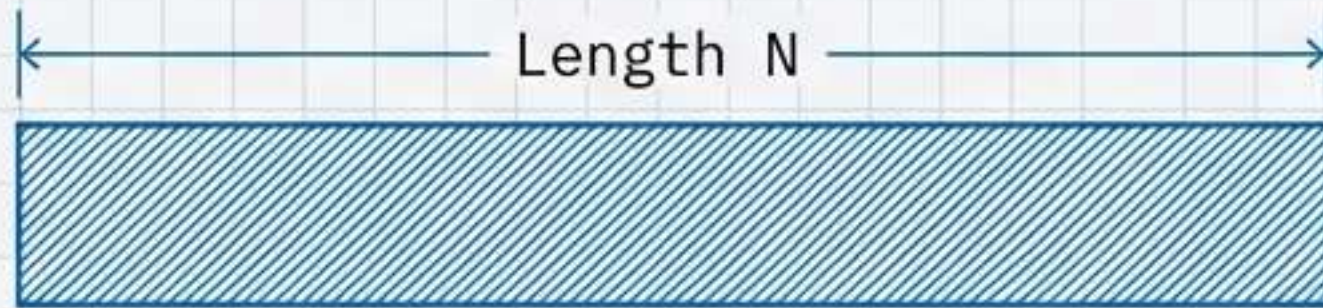
Implementation: Reverse Traversal (Recommended)

```
reverse_traversal.py × + _ □ ×  
  
def last_word_starts_with_upper_case(sentence: str):  
    words = sentence.split()  
  
    for word in reversed(words): # Scan Right -> Left  
        if word[0].isupper():  
            return word # Return immediately  
  
    return None
```

Optimized: Exits loop
on first match.

Logic Comparison

Scan All



Traverse entire list (Length N).
Constant updates.

Find & Exit



Return on first match.
Best case $O(1)$.



Winner: Reverse Traversal for cleaner logic.

The Final Solution

```
def last_word_starts_with_upper_case(sentence: str):  
    """  
    Finds the last word starting with an uppercase letter.  
    """  
    words = sentence.split()  
  
    for word in reversed(words):  
        if word[0].isupper():  
            return word  
  
    return None
```

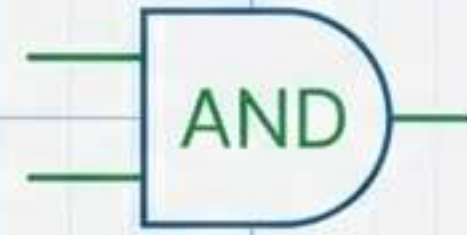
→ Ready for submission.
Includes docstring and robust None handling.

Key Takeaways



String Manipulation

Use `.split()` to parse sentences into lists.



Conditionals

Use `.isupper()` for character validation.



Algorithmic Thinking

Reverse Traversal optimizes "last item" searches.



Robustness

Always plan for the "None" case.

Time Complexity: $O(n)$ worst case. Logic Quality: High.